

EXPERIMENT: CLASS DIAGRAM DESIGN

1. Aim

To design and understand a class diagram representing the structure of a campus event management system, including event updates, student registration, and QR code-based access.

2. Objective

- To identify the main classes in the event management system
- To define attributes and methods of each class
- To understand relationships between classes
- To represent system structure using UML notation
- To model interactions like event creation, registration, and QR verification
- To study how data flows between different system components

3. Definition of Class Diagram

A class diagram is a type of UML (Unified Modeling Language) diagram that represents the static structure of a system.

It shows:

- Classes and their attributes
- Methods/functions of each class
- Relationships between classes (association, composition, etc.)
- Multiplicity (one-to-many, many-to-many relationships)

Class diagrams are widely used for designing object-oriented systems and understanding system architecture.

4. Procedure

Step 1: Identify Classes

From the system, the following classes are identified:

- Student

- Admin
- Event
- Registration
- QRCode
- Notification
- Scanner

Step 2: Define Attributes and Methods

Student

- Attributes: studentID, name, email, department
- Methods: login(), viewEvents(), registerEvent()

Admin

- Attributes: adminID, name, role
- Methods: createEvent(), updateEvent(), deleteEvent(), manageRegistrations()

Event

- Attributes: eventID, title, description, date, time, location, category
- Methods: createEvent(), updateEvent(), displayEvent()

Registration

- Attributes: registrationID, studentID, eventID, status
- Methods: register(), cancelRegistration()

QRCode

- Attributes: qrID, codeData, generatedTime
- Methods: generateQR(), validateQR()

Notification

- Attributes: notificationID, message, date
- Methods: sendNotification()

Scanner

- Methods: scanQR(), verifyEntry()

Step 3: Identify Relationships

- Student → Event
(Student views and registers for events)
- Student → Registration
(A student can have multiple registrations)
- Event → Registration
(An event can have multiple participants)
- Registration → QRCode
(Each registration generates a unique QR code)
- Admin → Event
(Admin creates and manages events)
- Admin → Registration
(Admin manages registrations)
- Notification → Student
(Students receive event updates)
- Scanner → QRCode
(Scanner verifies QR codes during entry)

Step 4: Define Multiplicity

- One Student → Many Registrations (1 to 0..*)
- One Event → Many Registrations (1 to 0..*)
- One Registration → One QR Code (1 to 1)
- One Admin → Many Events (1 to 0..*)
- One Student → Many Notifications (1 to 0..*)

Step 5: Add Relationships in Diagram

- Use association lines between classes
- Use arrows to indicate direction
- Label relationships such as:
 - registers
 - creates
 - manages
 - generates

- scans
- Use diamond symbol for composition:
 - Registration → QRCode

Step 6: Finalize Diagram

- Ensure proper alignment of classes
- Clearly separate attributes and methods
- Add visibility symbols (+ for public)
- Maintain readability and spacing

5. Notations Used in the Diagram

Notation	Meaning
Class Box	Represents a class with attributes and methods
+ Symbol	Public visibility
Attributes	Variables of a class
Methods	Functions of a class
Association Line	Relationship between classes

Arrow →	Direction of relationship
Multiplicity (1, 0.., 1..)	Number of instances
Diamond (◆)	Composition relationship
Dashed Arrow	Dependency

6. Procedure to Create Sequence Diagram in Draw.io

Step 1: Open Draw.io

- Go to <https://draw.io> (or diagrams.net)
- Select **Device** or **Google Drive** to save your file

Step 2: Create a New Diagram

- Click on “**Create New Diagram**”
- Choose **Blank Diagram**
- Give a name (e.g., *Class Diagram*)
- Click **Create**

Step 3: Enable UML Shapes

- On the left panel, click “**More Shapes**”
- Tick **UML → UML Sequence**
- Click **Apply**

Step 4: Add Classes

Drag **Class** shapes and create:

User, Admin, Report, Evidence, SOSAlert, AIAalyzer, Dashboard

Step 5: Add Attributes & Methods

Divide each class into 3 sections:

- Class Name
- Attributes
- Methods

Enter details accordingly

Step 6: Draw Relationships

Use connectors to link classes

Label relationships:

- creates
- manages
- triggers
- handles
- analyzes

Step 7: Add Multiplicity

Add values near connectors:

- 1
- 0..*
- 1..*

Step 8: Add Composition

Use diamond symbol between:

Report and Evidence

Step 9: Final Formatting

Align all elements

Maintain equal spacing

Ensure readability

Step 10: Export Diagram

- Go to File → Export As → PNG/JPEG/PDF

- Save the file to your system

7. Procedure to Upload Diagram in Azure DevOps Wiki

Step 1: Open Azure DevOps

- Go to your project in **Azure DevOps**
- Navigate to **Wiki section**

Step 2: Open/Create Page

- Click on your wiki (e.g., *S.P.E.A.K-project wiki*)
- Select existing page or click **New Page**
- Give title (e.g., *Class Diagram*)

Step 3: Enter Edit Mode

- Click **Edit** button on top right

Step 4: Upload Image

- Drag and drop your exported diagram directly into editor
OR
- Click **Insert Image** option
- Upload from your system

Step 5: Adjust Image

- Resize if needed
- Add caption (optional)

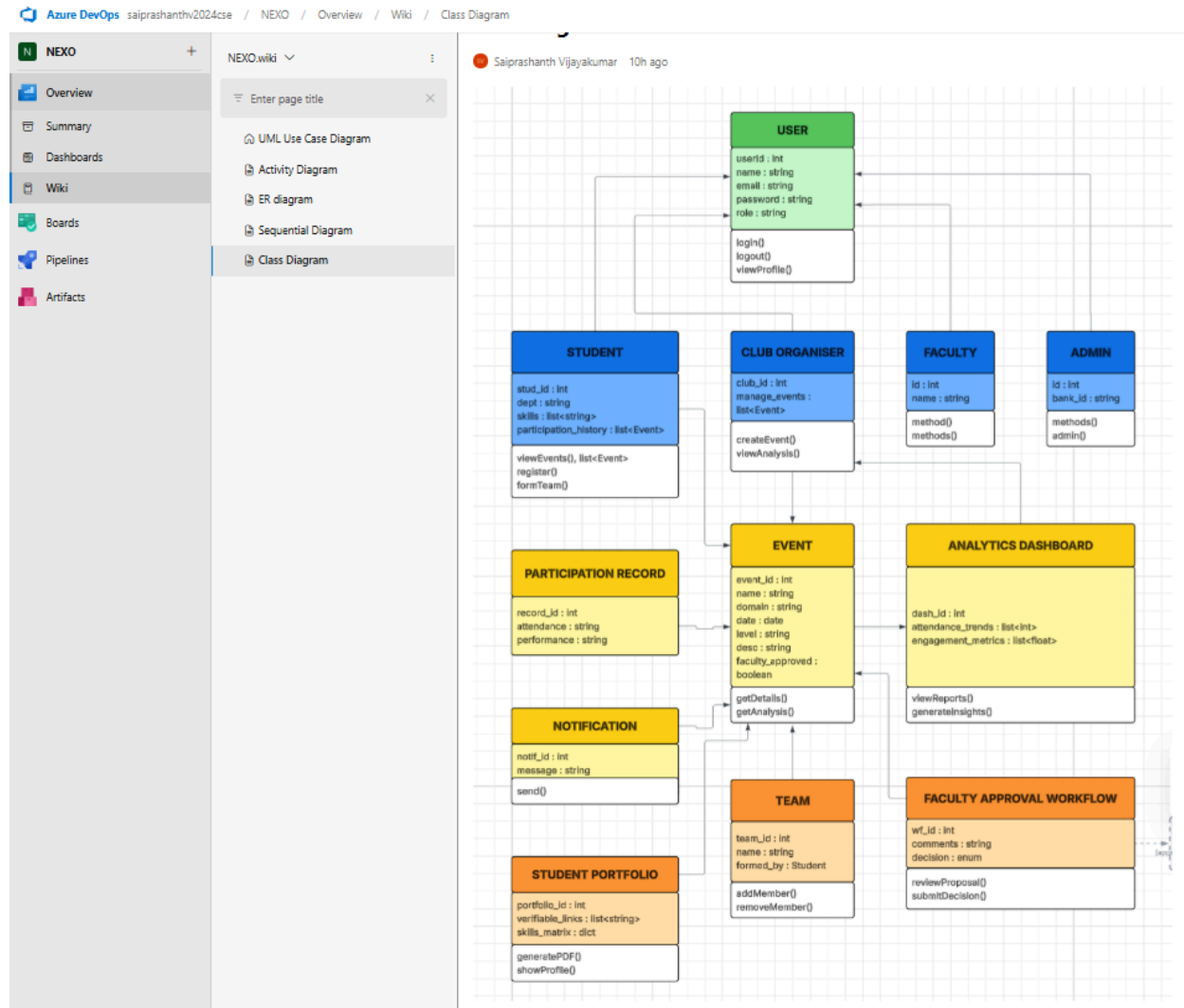
Step 6: Save Page

- Click **Save** or **Save & Close**
- Add commit message (e.g., *Added class diagram*)

Step 7: Verify Upload

- Check if diagram is visible properly
- Ensure clarity and readability

7. Outcome



8. Result

The class diagram was successfully created to represent the static structure of the campus event management system. It clearly illustrates the relationships between students, admins, events, registrations, QR codes, and notification mechanisms.

The diagram effectively models how different components interact, including event creation, student registration, QR code generation, and entry verification. This provides a clear understanding of the system architecture and supports efficient object-oriented design and implementation.